

GoogleQRCodeController

1. 簡介

GoogleQRCodeController 是一個 Spring MVC 控制器，負責根據用戶提供的數據生成 QR Code，並將其轉換為 Base64 格式的圖像，然後在檢視 (qrvue) 中顯示。

2. 程式碼解釋

2.1 控制器標註與請求對應

```
@Controller
@RequestMapping("/qrcode")
```

- @Controller 標註該類別為 Spring MVC 控制器，負責處理 Web 請求。
- @RequestMapping("/qrcode")：指定所有以 /qrcode 開頭的請求都會被此控制器處理。

2.2 QR Code 生成端點

```
@GetMapping("/generate")
public ModelAndView generateQRCode(@RequestParam("qrdata") String data,
Model model) throws WriterException {
```

- @GetMapping("/generate")：指定此方法處理 /qrcode/generate 的 HTTP GET 請求。
- @RequestParam("qrdata") String data：從請求參數 qrdata 取得用戶輸入的數據，作為 QR Code 的內容。
- Model model：Spring MVC 提供的 Model 物件，用來傳遞數據到視圖。

2.3 生成 QR Code

```
String qrCodeBase64 = generateQRCodeImage(data);
model.addAttribute("qrCode", qrCodeBase64);
model.addAttribute("codeString", data);
```

- `generateQRCodeImage(data)`: 呼叫 `generateQRCodeImage` 方法, 將 `data` 轉換成 QR Code 圖像並編碼為 Base64。
- `model.addAttribute("qrCode", qrCodeBase64)`: 將 QR Code 的 Base64 字串存入 Model, 供視圖使用。
- `model.addAttribute("codeString", data)`: 將原始數據存入 Model, 供視圖顯示。

2.4 返回視圖

```
return new ModelAndView("qrview");
```

- `new ModelAndView("qrview")`: 指定返回的視圖名稱 `qrview`, 這通常對應到 `qrview.html` 或 `qrview.jsp`。

3. QR Code 生成方法

```
private String generateQRCodeImage(String data) throws WriterException
{
```

此方法負責生成 QR Code 圖像, 並轉換為 Base64 格式。

3.1 產生 QR Code 的圖像數據

```
QRCodeWriter qrCodeWriter = new QRCodeWriter();
BitMatrix bitMatrix = qrCodeWriter.encode(data, BarcodeFormat.QR_CODE,
250, 250);
```

- `QRCodeWriter qrCodeWriter = new QRCodeWriter();`: 使用 `QRCodeWriter` 來生成 QR Code。
- `bitMatrix = qrCodeWriter.encode(data, BarcodeFormat.QR_CODE, 250, 250);`: 將 `data` 編碼為 250x250 的 QR Code 點陣數據。

3.2 將 BitMatrix 轉換為影像

```
BufferedImage image = new BufferedImage(250, 250,
BufferedImage.TYPE_INT_RGB);
for (int x = 0; x < 250; x++) {
    for (int y = 0; y < 250; y++) {
        image.setRGB(x, y, bitMatrix.get(x, y) ? 0x000000 : 0xFFFFFFFF);
    }
}
```

```
}
```

- BufferedImage 物件建立 250x250 的影像。
- 透過 for 迴圈將 bitMatrix 轉換為黑白像素點：
 - 0x000000 (黑色) 代表 QR Code 的內容。
 - 0xFFFFFFFF (白色) 代表背景。

3.3 轉換為 Base64

```
ByteArrayOutputStream baos = new ByteArrayOutputStream();  
ImageIO.write(image, "png", baos);  
return "data:image/png;base64," +  
Base64.getEncoder().encodeToString(baos.toByteArray());
```

- ByteArrayOutputStream baos = new ByteArrayOutputStream();：創建一個記憶體中的位元流。
- ImageIO.write(image, "png", baos);：將影像寫入 baos。
- Base64.getEncoder().encodeToString(baos.toByteArray());：將影像轉換為 Base64 編碼的字串。
- 最終回傳 Base64 字串，前綴 data:image/png;base64，以便前端直接渲染 QR Code 圖像。

3.4 錯誤處理

```
catch (Exception e) {  
    return "無法產生 QR Code";  
}
```

如果在轉換過程中發生錯誤，則返回錯誤訊息。

4. 如何使用

1. 啟動 Spring Boot 應用程式。
2. 在瀏覽器輸入以下 URL，將 HelloWorld 替換為任意字串：
3. <http://localhost:8080/qrcode/generate?qrdata=HelloWorld>
4. 瀏覽器會顯示 qrview 視圖，其中包含 QR Code 影像。

5. 小結

- 此控制器利用 QRCodeWriter 來生成 QR Code。
- 影像轉換為 Base64 字串，並可直接在 HTML 標籤中顯示。
- 透過 Spring MVC 的 @Controller 和 @RequestMapping 來處理請求。
- 這種方式適用於 Web 應用程式，不需要額外的靜態影像存儲。

pom.xml

```
<!-- ZXing Library for QR Code -->
<dependency>
    <groupId>com.google.zxing</groupId>
    <artifactId>core</artifactId>
    <version>3.5.1</version>
</dependency>
<dependency>
    <groupId>com.google.zxing</groupId>
    <artifactId>javase</artifactId>
    <version>3.5.1</version>
</dependency>
```

GoogleQRCodeController.java

```
import java.awt.image.BufferedImage;
import java.io.ByteArrayOutputStream;
import java.util.Base64;
```

```
import javax.imageio.ImageIO;
```

```
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.servlet.ModelAndView;
```

```
import com.google.zxing.BarcodeFormat;
import com.google.zxing.WriterException;
import com.google.zxing.common.BitMatrix;
import com.google.zxing.qrcode.QRCodeWriter;
```

```

@Controller
@RequestMapping("/qrcode")
public class GoogleQRCodeController {

    @GetMapping("/generate")
    public ModelAndView generateQRCode(@RequestParam("qrdata") String data,
    ModelAndView model) throws WriterException {
        String qrCodeBase64 = generateQRCodeImage(data);
        model.addAttribute("qrCode", qrCodeBase64);
        model.addAttribute("codeString", data);

        return new ModelAndView("qrview");
    }

    private String generateQRCodeImage(String data) throws WriterException {
        QRCodeWriter qrCodeWriter = new QRCodeWriter();
        BitMatrix bitMatrix = qrCodeWriter.encode(data, BarcodeFormat.QR_CODE,
        250, 250);
        BufferedImage image = new BufferedImage(250, 250,
        BufferedImage.TYPE_INT_RGB);
        for (int x = 0; x < 250; x++) {
            for (int y = 0; y < 250; y++) {
                image.setRGB(x, y, bitMatrix.get(x, y) ? 0x000000 : 0xFFFFFF);
            }
        }
        try {
            ByteArrayOutputStream baos = new ByteArrayOutputStream();
            ImageIO.write(image, "png", baos);
            return "data:image/png;base64," +
            Base64.getEncoder().encodeToString(baos.toByteArray());
        } catch (Exception e) {
            return "無法產生 QR Code";
        }
    }
}

```

qrview.jsp

```
<%@ page language="java" contentType="text/html; charset=UTF-8"
    pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
<head>
    <meta charset="UTF-8">
    <title>Google QR Code</title>
</head>
<body>
    <h2>掃描 QR Code </h2>
    
    <h2>
    <p>QR Code 內容： <label>{codeString}</label></p>

    </h2>
</body>
</html>
```